Rule Engine — API Reference

Runtime rule management & validation. Library `com.leads:rule-engine:3.0.0` (4-table design).

Base URL `http://localhost:8080` · All bodies `application/json` · Author K M Farhat Snigdah

1. Overview

The rule engine is a Spring Boot **starter library**. A consuming service adds the dependency and gets two things: a runtime **validation call** its code invokes, and an **Admin REST API** (under `/admin`) to manage flows, groups and rules while the app runs — no restart, no redeploy.

Routing is database-driven. An incoming **request class** maps 1:1 to a **flow**; a flow runs an ordered list of **agenda groups** (shared `COMMON` first, then the flow's own); each group holds **rules**. A rule is stored as both an editable builder form (`source_json`) and the executable `drl_text` the engine compiles.

Data model (4 tables)

TABLE	HOLDS	KEY POINTS
`rule_flow`	flow master + router	`request_class` (UNIQUE, routing key) and `fact_type` live here; has `active`
`rule_group`	a flow's agenda groups	`flow_id` NULL = shared (COMMON); `UNIQUE(flow_id, group_code)`; has `active`
`rule_definition`	the rules	FK to one group; `UNIQUE(group_id, rule_name)`; keeps both `source_json` & `drl_text`; has `active`
`rule_field_catalog`	builder field palette	`flow_id` NULL = shared field

Conventions

- **Enablement:** the Admin API is exposed only when `rule-engine.admin.enabled=true` (the default). Secure this path in production (e.g. the AuthZ library).
- **Identity:** flows are addressed by `flowCode`; groups and rules by numeric `id`.
- **Immediate effect:** every write reloads routing and/or the rule base synchronously.

Error model

Admin errors return a JSON envelope. `400` — bad input (unknown id, missing `sourceJson`/`drl`, DRL compile error, unregistered request class, bean-validation). `409` — deleting a flow/group that still has children without `?force=true`.

```
// 400 — validation
{ "timestamp":"2026-10-13T…Z", "status":400, "error":"Bad Request",
  "message":"Validation failed", "fieldErrors": { "factType":"must not be blank" } }

// 409 — group still has rules
{ "timestamp":"…", "status":409, "error":"Conflict",
  "message":"Group 4 ('COMPLIANCE') has 2 rule(s). Delete them first, or call again with ?
force=true." }
```

2. Active / inactive & delete

Every flow, group and rule has two independent operations:

- **Activate / deactivate** — the UI on/off toggle. Keeps the row; flips `active`. An inactive **rule** is not compiled, an inactive **group** is not focused, and an inactive **flow** runs nothing at all when its request hits (the request simply passes through — not an error).
- **Delete** — removes the row permanently. `?force=true` cascades to children (a flow's groups/fields/rules; a group's rules).

> **Deactivate ≠ delete.** Deactivating a COMPLIANCE group turns the screen off but keeps its rules; toggle it back on and they fire again. Deleting removes them for good.

3. Admin API — Flows

GET `/admin/flows`

List every flow with the groups it runs (shared first, then its own, in order). Includes `active` flags.

200 RESPONSE — FLOWVIEW[]

```
[ { "id":1, "flowCode":"TRANSFER_TRANSACTION",
    "requestClass":"com.example.droolspoc.dto.TransactionRequest",
    "factType":"com.example.droolspoc.model.ValidationContext",
    "label":"Transfer Transaction", "active":true,
    "groups":[
      { "id":1,"groupCode":"COMMON","orderNo":1,"shared":true,"active":true },
      { "id":2,"groupCode":"TRANSFER","orderNo":2,"shared":false,"active":true } ] } ]
```

POST `/admin/flows`

Create or update a flow master (matched by `flowCode`). Any `groups` listed are ensured to exist for the flow, in order (append-only — never deletes existing groups or rules).

REQUEST — FLOWDEFINITIONREQUEST

```
{ "flowCode":"TRANSFER_TRANSACTION",          // required
  "requestClass":"com.example.droolspoc.dto.TransactionRequest",
  "factType":"com.example.droolspoc.model.ValidationContext",  // required
  "label":"Transfer Transaction",
  "groups":["TRANSFER","VELOCITY","COMPLIANCE"] }
```

200 RESPONSE

The created/updated `FlowView`.

PUT `/admin/flows/{flowCode}/active`

Activate or deactivate a flow. An inactive flow runs no rules when its request hits.

REQUEST — ACTIVEREQUEST

```
{ "active": false }
```

200 RESPONSE

The updated `FlowView`.

DELETE `/admin/flows/{flowCode}`

Permanently delete a flow. `409` if it still has groups or fields, unless `?force=true` (which also deletes its groups, their rules, and its fields).

QUERY

`force` — boolean, default `false`.

RESPONSES

`204` deleted · `409` has children · `400` no such flow.

4. Admin API — Groups

GET `/admin/flows/{flowCode}/groups`

The groups a flow runs: shared groups first, then the flow's own, in order (active and inactive). Use the returned ids to attach rules.

200 RESPONSE — GROUPVIEW[]

```
[ { "id":1,"groupCode":"COMMON","orderNo":1,"shared":true,"active":true },
  { "id":4,"groupCode":"COMPLIANCE","orderNo":4,"shared":false,"active":true } ]
```

POST `/admin/flows/{flowCode}/groups`

Add a group to a flow. `orderNo` optional — appended after existing groups when omitted.

REQUEST — GROUPREQUEST

```
{ "groupCode":"COMPLIANCE", "orderNo": 4 }
```

200 RESPONSE

The created `GroupView`. `400` if the flow already has that group code.

POST `/admin/groups/shared`

Create a **shared** group (`flow_id NULL`, e.g. COMMON) that every flow runs first. There is one shared group per code.

REQUEST — GROUPREQUEST

```
{ "groupCode":"COMMON", "orderNo": 1 }
```

200 RESPONSE

The created `GroupView` (`shared:true`). `400` if that shared code already exists.

PUT `/admin/flows/{flowCode}/groups/order`

Rewrite the firing order of a flow's own groups from the given id order.

REQUEST — GROUPORDERREQUEST

```
{ "groupIds": [2, 4, 3] }
```

200 RESPONSE

Empty. `400` if an id doesn't belong to the flow.

PUT `/admin/groups/{groupId}/active`

Activate or deactivate a group. An inactive group is skipped; its rules are not evaluated.

REQUEST — ACTIVEREQUEST

```
{ "active": false }
```

200 RESPONSE

The updated `GroupView`.

DELETE `/admin/groups/{groupId}`

Permanently delete a group. `409` if it still has rules, unless `?force=true` (deletes its rules too).

QUERY

`force` — boolean, default `false`.

RESPONSES

`204` deleted · `409` has rules.

5. Admin API — Rules

GET /admin/groups/{groupId}/rules

All rules in one exact group — active and inactive, so the UI can show and flip the toggle.

200 RESPONSE — RULEVIEW[]


```
[ { "id":7,"groupId":4,"groupCode":"COMPLIANCE","flowCode":"TRANSFER_TRANSACTION",
    "ruleName":"Block high-value cross-border non-BDT transfer",
    "sourceType":"BUILDER","active":true,"version":1,
    "sourceJson":"{…}","drlText":"package …" } ]
```


POST /admin/groups/{groupId}/rules

Create or replace a rule in a group (matched by `ruleName` within the group). **BUILDER** rules send `sourceJson` and the engine generates the DRL (only in a flow group — needs a fact type); **DRL** rules send `drl` stored as-is (the only option for the shared COMMON group). The candidate is compiled with the live set first — a rule that doesn't compile is rejected (`400`) and the running engine is untouched.

REQUEST — RULEDEFINITIONREQUEST

```
{ "ruleName":"Block …",            // required
  "sourceType":"BUILDER",          // BUILDER | DRL (required)
  "sourceJson":"{…tree…}",         // required for BUILDER (a JSON string)
  "drl":"package …; rule …end",    // required for DRL
  "createdBy":"farhat" }
```

200 RESPONSE

The saved `RuleView`.

GET /admin/rules/{id}

One rule by id. `400` if not found.

200 RESPONSE

A `RuleView` (includes `sourceJson` and `drlText`).

PUT /admin/rules/{id}

Update a rule by id. The rule's group does not change; renaming must stay unique within the group. Same body as create.

200 RESPONSE

The updated `RuleView`.

PUT `/admin/rules/{id}/active`

Activate or deactivate a rule without deleting it (the on/off radio). Inactive rules are not compiled into the rule base.

REQUEST — ACTIVEREQUEST

```
{ "active": false }
```

200 RESPONSE

The updated `RuleView`.

DELETE `/admin/rules/{id}`

Permanently deletes the rule (row removed) and reloads the rule base. To merely disable it, use the active toggle above.

RESPONSES

`204` deleted.

POST `/admin/rules/preview`

Generate the DRL for a builder spec and compile-check it — **nothing is saved**, the live engine is untouched. Use it to show the generated DRL and whether it is valid before saving.

REQUEST — BUILDERRULESPEC

```
{ "ruleName":"Block …", "flow":"TRANSFER_TRANSACTION", "agendaGroup":"COMPLIANCE",
  "salience":20, "conditionTree": { …see Appendix… },
  "then": { "decision":"block", "message":"…" },
  "otherwise": { "decision":"allow" }   // optional (emits a De-Morgan-negated 2nd rule) }
```

200 RESPONSE — RULEPREVIEWRESPONSE

```
{ "drl":"package …", "valid":true, "error":null }
```

6. Admin API — Fields & reload

GET `/admin/flows/{flowCode}/fields`

The builder field palette for a flow: the flow's own fields plus the shared ones (`flow_id NULL`), ordered by label. Read-only (the catalog is seeded in SQL).

200 RESPONSE — RULEFIELDCATALOG[]

```
[ { "id":2, "fieldKey":"transaction.amount", "label":"Transaction Amount",
    "dataType":"NUMBER", "drlPath":"transaction.amount",
    "allowedOps":"> < >= <= ==", "allowedValues":null } ]
```

POST /admin/fields

Add a builder field to the catalog. Omit (or blank) `flowCode` for a **shared** field shown in every flow; set it to scope the field to one flow.

REQUEST — FIELDREQUEST

```
{ "flowCode":"TRANSFER_TRANSACTION",   // optional — blank = shared
  "fieldKey":"transaction.amount",     // what the builder tree stores
  "label":"Transaction Amount",
  "dataType":"NUMBER",                 // NUMBER | ENUM | BOOLEAN | STRING
  "drlPath":"transaction.amount",      // real path in generated DRL
  "allowedOps":"> < >= <= ==",
  "allowedValues":null }               // for ENUM/BOOLEAN
```

200 RESPONSE

The created field (`RuleFieldCatalog`).

DELETE /admin/fields/{id}

Permanently delete a catalog field.

RESPONSES

`204` deleted.

POST /admin/flows/reload

Force a routing-cache refresh from the DB. Rarely needed (every write reloads) — use it after editing the DB directly.

200 RESPONSE

Empty.

7. Consumer integration (validation)

A consuming service does not call `/admin` to validate. It builds its context (a fact extending the flow's `fact_type`) and calls the engine, passing the **request class** as the routing key:

```
import leads.ruleengine.core.service.RuleExecutionService;

ruleExecutionService.execute(context, TransactionRequest.class, globalContext);
// rules write the outcome onto the context: isValid(), isPermissionDenied(),
getValidationMessage()
```

The POC wraps this behind three HTTP endpoints, shown here as a template for your own services. Response: `{ valid, permissionDenied, message }`.

POST **/api/transactions/validate**

```
{ "userId":"USER-001", "sourceAccount":"100001", "currency":"BDT", "amount":15000,
  "channel":"ONLINE", "country":"BD", "dailyTxnCount":1 }
// -> { "valid":false, "permissionDenied":true, "message":"Blocked: amount exceeds limit
…" }
```

POST **/api/loans/validate**

```
{ "userId":"USER-004", "amount":50000, "tenureMonths":12, "purpose":"Personal" }
```

POST **/api/account-closures/validate**

```
{ "userId":"USER-001", "accountNo":"ACC-901", "reason":"Customer request" }
```

8. Appendix — builder JSON shape

The `conditionTree` is a recursive AST. A **GROUP** joins children with its `operator` (`AND`/`OR`), nesting to any depth; a **CONDITION** is one comparison. `value` is a literal; `valueField` compares against another field instead.

```
"conditionTree": {
  "type":"GROUP", "operator":"OR",
  "children":[
    { "type":"GROUP","operator":"AND","children":[
        { "type":"CONDITION","field":"transaction.channel","operator":"==","value":"AGENT" },
        { "type":"CONDITION","field":"transaction.amount","operator":">","value":50000 } ] },
    {
"type":"CONDITION","field":"transaction.amount","operator":">","valueField":"userLimit.limit"
}
```

```
  ]
}
```

`field`/`valueField` are catalog `field_keys`; the generator resolves them to real `drl_paths` and types. The compiled Drools rule name is qualified `FLOW.GROUP::ruleName` so the same display name may exist in two groups.
